

AWS API Gateway Lab Part 1: Prerequisites

To Begin with the Lab

Summary of the Lab

In this first part of the **API Gateway** lab, the prerequisites were prepared before building the actual API routes. A **DynamoDB** table named **UsersTable** was created as the backend database with **userId** as the partition key and string type. The video then explained why an API is needed between the client app and DynamoDB, since mobile apps such as React Native or Flutter cannot talk to the table directly. Three separate **AWS Lambda** functions were planned for the core API logic: create user, get user, and delete user, following separation of concerns. Finally, an **IAM role** named **LambdaDynamoDBExecutionRole** was created with **AmazonDynamoDBFullAccess** so the Lambda functions could access DynamoDB securely. The PDF also confirms that this lab uses **HTTP API**, and that a later lab will cover the same flow using **REST API**.

Understanding the Prerequisites for API Gateway

API Gateway exists so a front-end application can safely communicate with backend services through HTTP endpoints instead of calling **DynamoDB** directly. In this lab, the backend logic is split into small **Lambda** functions because each function handles one task: create, fetch, or delete a user. That keeps the design simple, testable, and easier to maintain. **IAM roles** are needed because Lambda must be trusted and authorized before it can read from or write to DynamoDB.

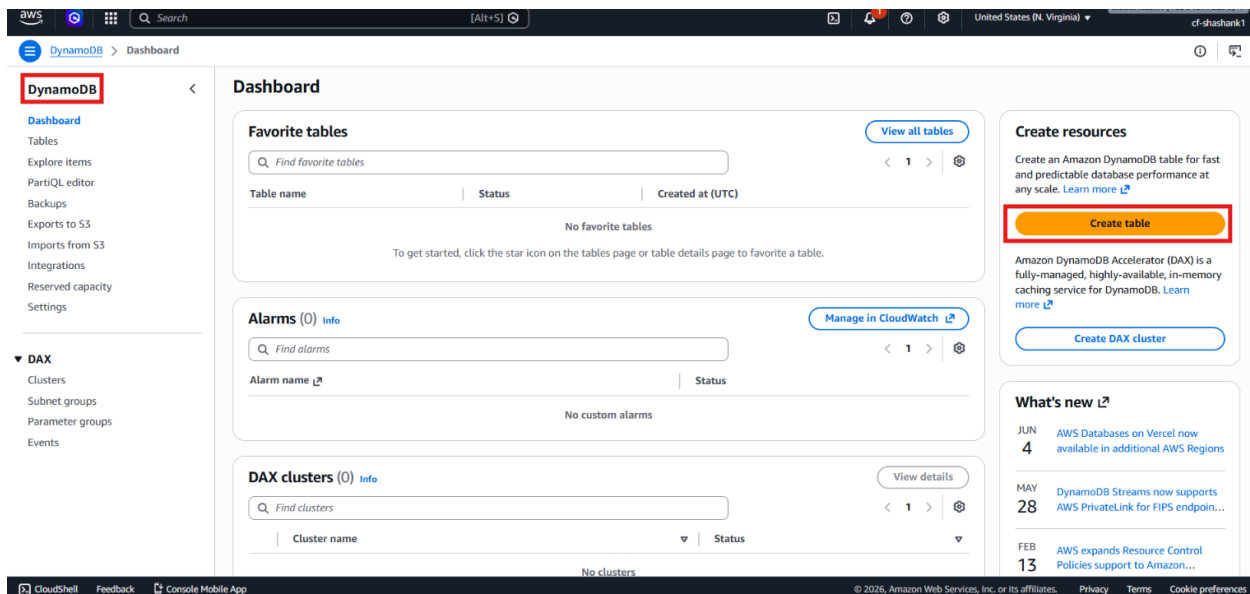
Example:

A Flutter app wants to save a new user record, fetch user details, and delete a user later. Instead of letting the app connect directly to **UsersTable**, the app sends requests to **API Gateway**, which triggers Lambda functions. Those Lambda functions then use the **LambdaDynamoDBExecutionRole** to interact with DynamoDB securely.

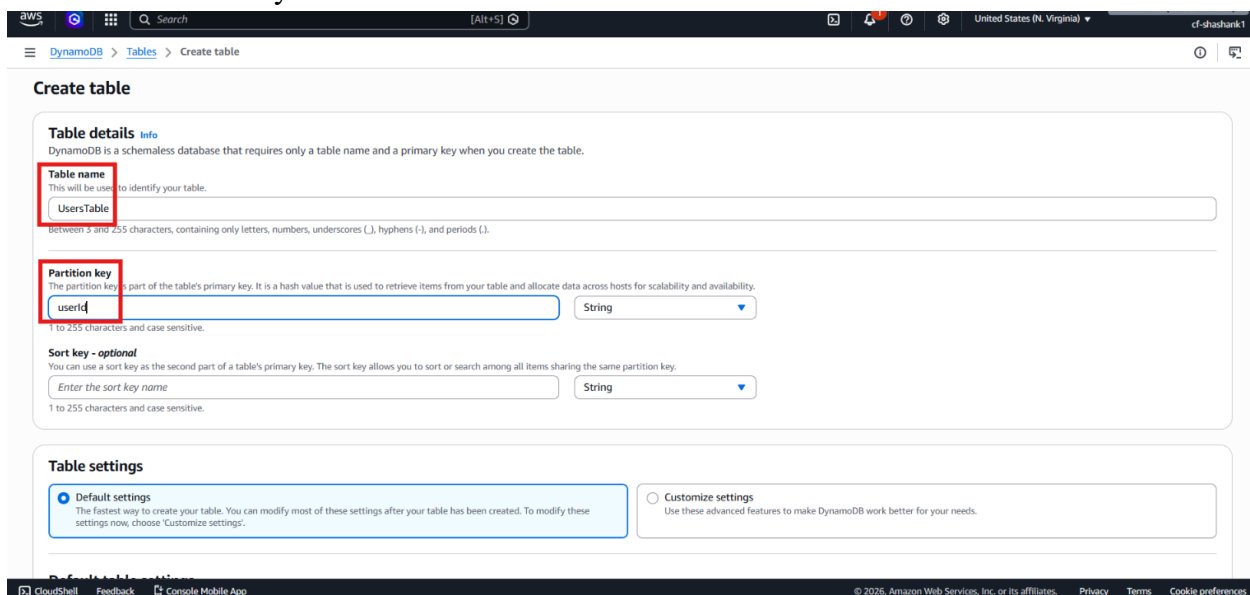
Lab Steps

Step 1 – Create the DynamoDB Backend Table

- Sign in to the AWS Management Console.
- Open **Amazon DynamoDB**.
- Click **Create table**.



- Enter the table name: UsersTable
- Set the partition key name to userId.
- Set the partition key type to String.
- Leave the sort key blank.



- Click **Create table**.

Step 2 – Understand the API Requirement

- Review the lab goal.
- Note that the front-end application cannot directly access DynamoDB.
- Understand that an API layer is needed for:
 - Creating a user
 - Fetching a user

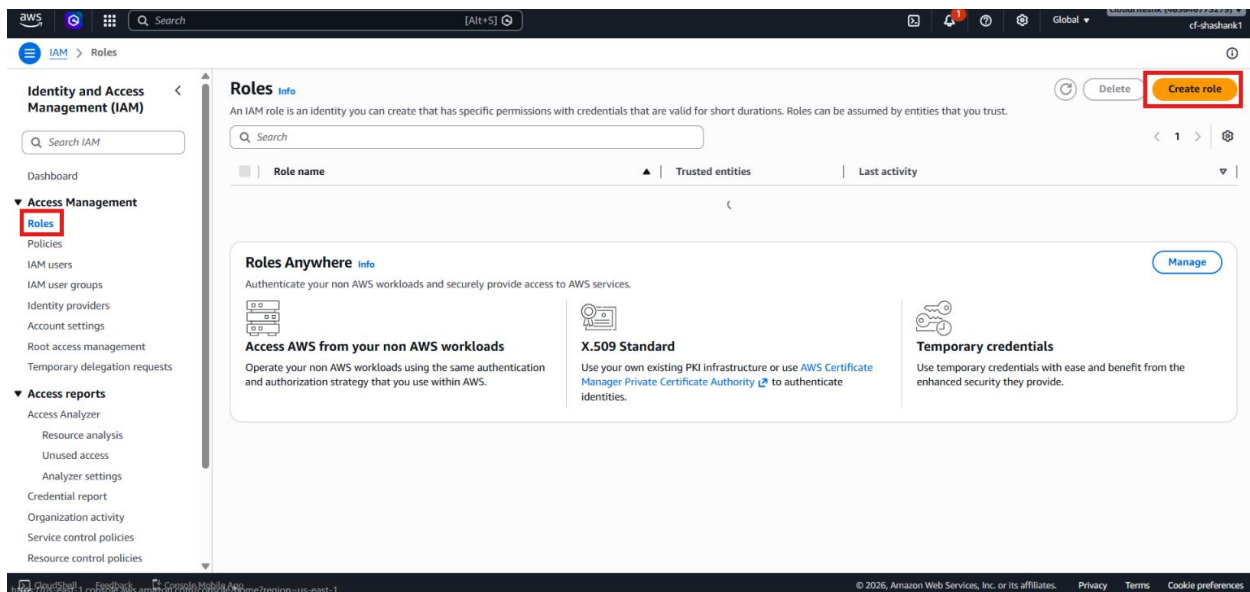
- Deleting a user
- Understand that the API logic will be hosted in **AWS Lambda**.

Step 3 – Plan the Lambda Functions

- Decide to create three separate Lambda functions:
 - createUserFunction
 - getUserFunction
 - deleteUserFunction
- Keep each function focused on one task.
- Follow separation of concerns for easier testing and maintenance.

Step 4 – Create the IAM Role for Lambda

- Open **IAM** → **Roles**.
- Click **Create role**.



- Choose **AWS service** as the trusted entity type.
- Select **Lambda**.

Step 2: Add permissions

Trusted entity type

- ☒ AWS service
- ☐ AWS account
- ☐ Web identity
- ☐ SAML 2.0 federation
- ☐ Custom trust policy

Use case

Service or use case: Lambda

Choose a use case for the specified service.

Use case: Lambda

Cancel Next

- Attach AmazonDynamoDBFullAccess.
- Enter the role name: LambdaDynamoDBExecutionRole

Step 2: Add permissions

Add permissions

Permissions policies (1/2300)

Choose one or more policies to attach to your new role.

Filter by Type: All types, 4 matches

Policy name	Type	Description
☒ AmazonDynamoDBFullAccess	AWS managed	Provides full access to Amazon DynamoDB...
☐ AmazonDynamoDBFullAccess_v2	AWS managed	Provides full access to Amazon DynamoDB
☐ AmazonDynamoDBFullAccesswithDataPipeline	AWS managed	This policy is on a deprecation path. See ...
☐ AmazonDynamoDBReadOnlyAccess	AWS managed	Provides read only access to Amazon Dyn...

Set permissions boundary - optional

Cancel Previous Next

- Create the role.
- Save this role for all three Lambda functions.

Step 5 – Confirm the Prerequisites Are Ready

- Verify that the **UsersTable** DynamoDB table exists.
- Verify that the **LambdaDynamoDBExecutionRole** has been created.
- Confirm that Lambda now has permission to talk to DynamoDB.
- Prepare to create the HTTP API in the next part of the lab.

Verification

- The **UsersTable** DynamoDB table was created successfully.

- The partition key `userId` was configured as a string.
- The API use case was understood.
- Three Lambda functions were planned for separate CRUD actions.
- The **LambdaDynamoDBExecutionRole** was created with DynamoDB access.
- The prerequisites for the next API Gateway part were completed.